고급통계분석_Assignment1

20220912_수학_이연우

2026-10-13

Problem 1

(a)

데이터 생성

```
set.seed(1)
n = 200
x = seq(0, 1, length.out = n)
y = sin(2 * pi * x) + rnorm(n, sd = 0.15)
df = data.frame(x = x, y = y)

write.csv(df, "data/problem1.csv", row.names = FALSE)
```

(b)

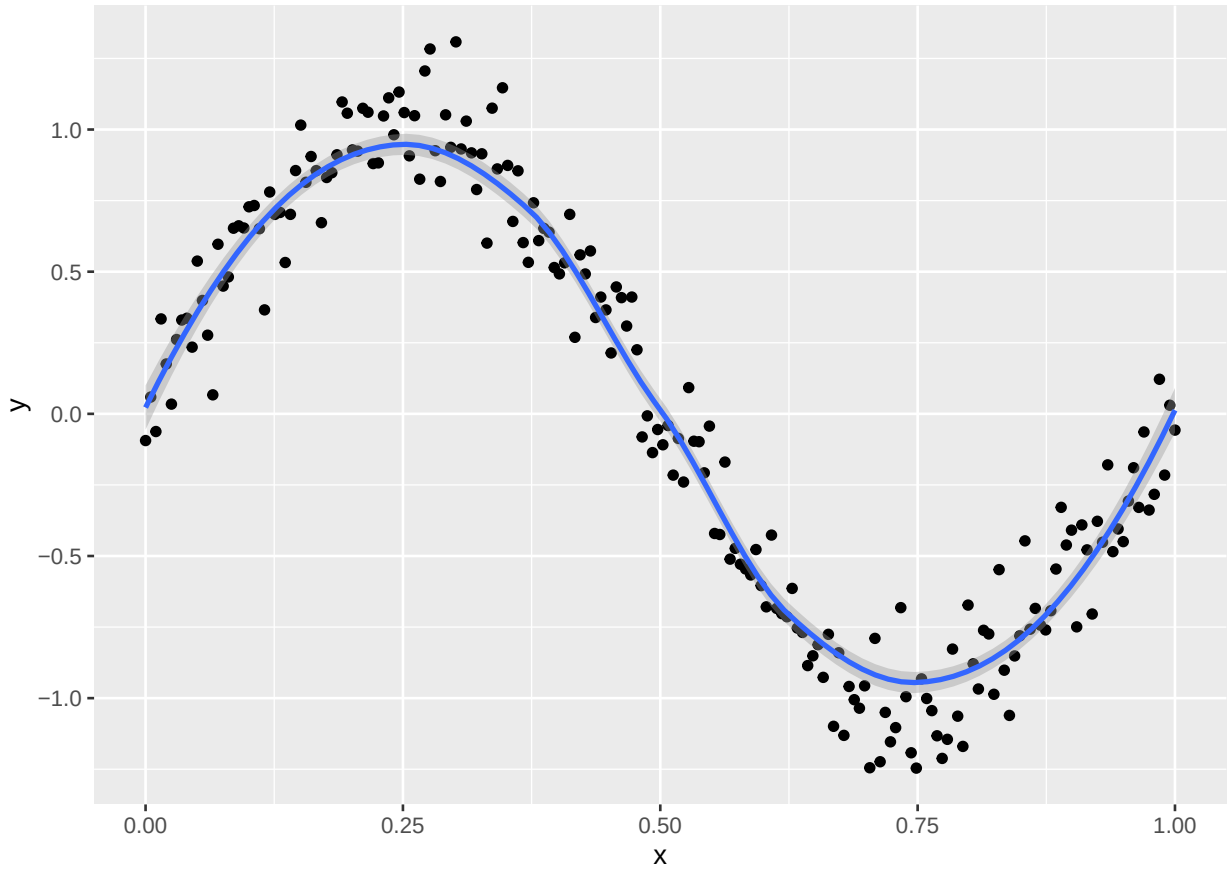
데이터 불러오기 및 시각화

```
library(ggplot2)
df = read.csv("data/problem1.csv")
head(df)
```

```
##             x           y
## 1 0.000000000 -0.09396807
## 2 0.005025126  0.05911505
## 3 0.010050251 -0.06223866
## 4 0.015075377  0.33387193
## 5 0.020100503  0.17538587
## 6 0.025125628  0.03414379
```

```
p = ggplot(df, aes(x = x, y = y)) +
  geom_point() +
  geom_smooth(method = "loess")

print(p)
```



(c)

그래프 저장

```
ggsave("plots/problem1.png", plot = p)
```

(d)

https://github.com/ywlee15/Assignment1

Problem 2

(a)

```
bubble_sort = function(vec, decreasing = FALSE)
{
  n = length(vec)
```

```
  if (n < 2) return(vec)

  for (i in 1:(n-1))
  {
    for(j in 1:(n-i))
    {
      if (decreasing == FALSE)
      {
        if (vec[j] > vec[j+1])
        {
          temp = vec[j]
          vec[j] = vec[j+1]
          vec[j+1] = temp
        }
      }
      else
      {
        if (vec[j] < vec[j+1])
        {
          temp = vec[j]
          vec[j] = vec[j+1]
          vec[j+1] = temp
        }
      }
    }
  }
  return(vec)
}

set.seed(1)
x = runif(10)
bubble_sort(x, decreasing = FALSE)
```

```
##  [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336 0.62911404
##  [7] 0.66079779 0.89838968 0.90820779 0.94467527
```

```
bubble_sort(x, decreasing = TRUE)
```

```
##  [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404 0.57285336
##  [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

(b)

```
quick_sort = function(vec, decreasing = FALSE)
{
  n = length(vec)

  if (n < 2) return (vec)
```

```
  pivot = vec[1]

  left = c()
  right = c()

  for (j in 2:n)
  {
    if (decreasing == FALSE)
    {
      if (vec[j] <= pivot)
      {
        left = c(left, vec[j])
      }
      else
      {
        right = c(right, vec[j])
      }
    }
    else
    {
      if (vec[j] >= pivot)
      {
        left = c(left, vec[j])
      }
      else
      {
        right = c(right, vec[j])
      }
    }
  }
  left = quick_sort(left, decreasing)
  right = quick_sort(right, decreasing)

  return(c(left, pivot, right))
}

set.seed(1)
x = runif(10)
quick_sort(x, decreasing = FALSE)
```

```
##  [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336 0.62911404
##  [7] 0.66079779 0.89838968 0.90820779 0.94467527
```

```
quick_sort(x, decreasing = TRUE)
```

```
##  [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404 0.57285336
##  [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

Problem 3

(a)

```
numerical_diff = function(f, x, h = 1e-6, method = "central")
{
  if (method == "forward")
  {
    result = (f(x+h) - f(x)) / h
  }
  else if (method == "backward")
  {
    result = (f(x) - f(x-h)) / h
  }
  else if (method == "central")
  {
    result = (f(x+h) - f(x-h)) / (2*h)
  }
  return(result)
}

f = function(x)
{
  return(cos(x) - x)
}

f_prime = function(x)
{
  return(-sin(x) - 1)
}



x = seq(0, 2*pi, length.out = 100)

y_forward = numerical_diff(f, x, method = "forward")
y_backward = numerical_diff(f, x, method = "backward")
y_central = numerical_diff(f, x, method = "central")

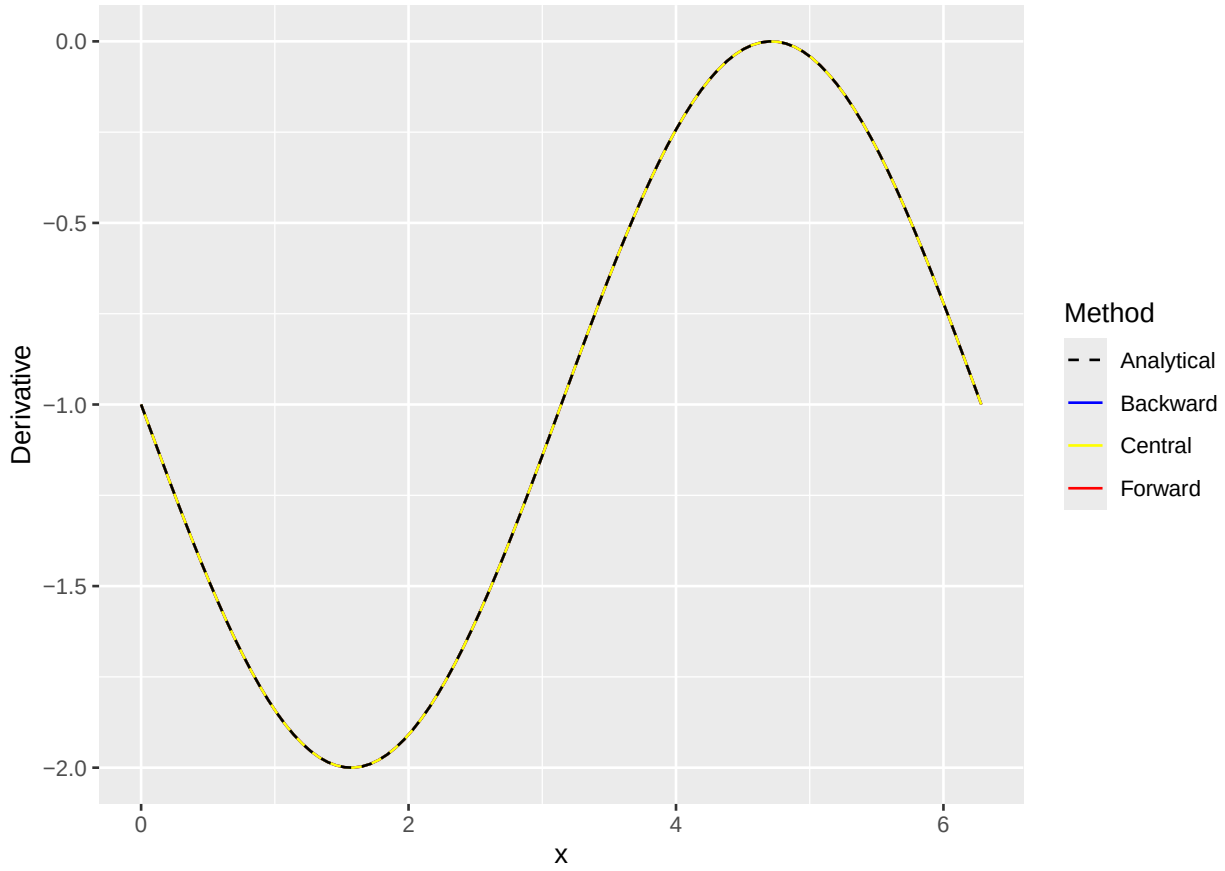

y_true = f_prime(x)


library(ggplot2)

df = data.frame(
  x = x,
  Forward = y_forward,
  Backward = y_backward,
  Central = y_central,
  Analytical = y_true
)
```

```r
p = ggplot(df, aes(x = x, group = 1)) +
  geom_line(aes(y = Forward, color = "Forward")) +
  geom_line(aes(y = Backward, color = "Backward")) +
  geom_line(aes(y = Central, color = "Central")) +
  geom_line(aes(y = Analytical, color = "Analytical"),
            linetype = "dashed") +
  scale_color_manual(values = c(
    "Forward" = "red",
    "Backward" = "blue",
    "Central" = "yellow",
    "Analytical" = "black"
  )) +
  labs(x = "x", y = "Derivative", color = "Method")

print(p)
```




```r
ggsave("plots/problem3.png", plot = p)
```

(b)

```
newtown_rapshon = function(f, fprime = NULL, x0, maxiter = 100, h = 1e-6, epsilon = 1e-10)
{
  x_t_1 = x0

  for (t in 1:maxiter)
  {
    if(is.null(fprime))
    {
      derivative = numerical_diff(f, x_t_1, h = h, method = "central")
    }
    else
    {
      derivative = fprime(x_t_1)
    }

    # Newton-Rapshon
    x_t = x_t_1 - f(x_t_1) / derivative

    if (abs(x_t - x_t_1) < epsilon)
    {
      return(x_t)
    }

    x_t_1 = x_t
  }
  return(x_t_1)
}
```

(c)

```
root_numerical = newtown_rapshon(f, x0 = 0.5)
root_numerical
```

```
## [1] 0.7390851
```

```
root_analytical = newtown_rapshon(f, fprime = f_prime, x0 = 0.5)
root_analytical
```

```
## [1] 0.7390851
```

Problem 4

(a)

```
left_rectangle = function(f, a, b, n)
{
  h = (b - a) / n
  result = 0

  for (i in 0:(n-1))
  {
    x_i = a + i * h
    result = result + f(x_i)
  }
  result = h * result

  return (result)
}
```

(b)

```
trapezoid = function(f, a, b, n)
{
  h = (b - a) / n
  result = f(a) + f(b)

  for(i in 1:(n-1))
  {
    x_i = a + i * h
    result = result + 2 * f(x_i)
  }

  result = (h/2) * result

  return(result)
}
```

(c)

```
simpson = function(f, a, b, n)
{
  if (n %% 2 != 0)
  {
    stop("n must be even")
  }

  h = (b - a) / n
  result = f(a) + f(b)

  for (i in 1:(n-1))
  {
    x_i = a + i * h
```

```
    if (i %% 2 == 1)
    {
      result = result + 4 * f(x_i)
    }
    else
    {
      result = result + 2 * f(x_i)
    }
  }
  result = (h/3) * result

  return(result)
}
```

(d)

```
## 1. Left Rectangle
left_rectangle(sin, a = 0, b = pi, n = 100)
```

```
## [1] 1.999836
```

```
## 2. Trapezoid
trapezoid(sin, a = 0, b = pi, n = 100)
```

```
## [1] 1.999836
```

```
## 3. Simpson
simpson(sin, a= 0, b = pi, n = 100)
```

```
## [1] 2
```

(e)

```
n_values = c(10, 30, 60, 100, 150, 200)
true_value = -cos(pi) + cos(0)

error_left = numeric(length(n_values))
error_trapezoid = numeric(length(n_values))
error_simpson = numeric(length(n_values))

for (i in 1:(length(n_values)))
{
  n = n_values[i]

  error_left[i] = abs(left_rectangle(sin, 0, pi, n) - true_value)
  error_trapezoid[i] = abs(trapezoid(sin, 0, pi, n) - true_value)
  error_simpson[i] = abs(simpson(sin, 0, pi, n) - true_value)
```

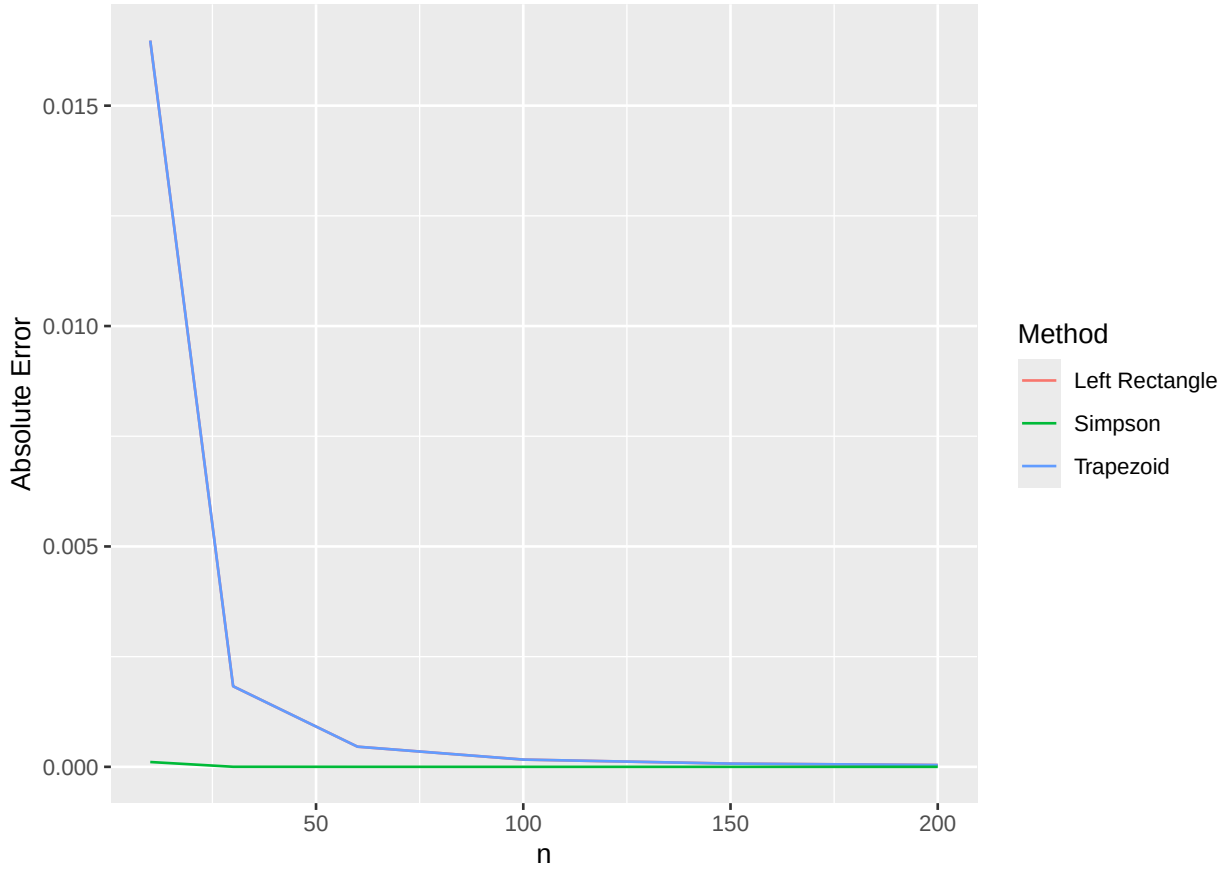
```
}

df = data.frame(
  n = n_values,
  Left = error_left,
  Trapezoid = error_trapezoid,
  Simpson = error_simpson
)

library(ggplot2)

p = ggplot(df, aes(x = n, group = 1)) +
  geom_line(aes(y = Left, color = "Left Rectangle")) +
  geom_line(aes(y = Trapezoid, color = "Trapezoid")) +
  geom_line(aes(y = Simpson, color = "Simpson")) +
  labs(x = "n", y = "Absolute Error", color = "Method")

print(p)
```




```
ggsave("plots/problem4.png", plot = p)
```

Problem 5

(a)

```
A = matrix(c(4, 2, 2,
             2, 5, 1,
             2, 1, 3), nrow = 3)
L = t(chol(A))
print(L)
```

```
##      [,1] [,2]     [,3]
## [1,]    2    0 0.000000
## [2,]    1    2 0.000000
## [3,]    1    0 1.414214
```

```
print(L%*%t(L))
```

```
##      [,1] [,2] [,3]
## [1,]    4    2    2
## [2,]    2    5    1
## [3,]    2    1    3
```

(b)

```
forward = function(L, b)
{
  n = length(b)
  z = numeric(n)

  for (i in 1:n)
  {
    result = b[i]

    if (i > 1)
    {
      for (j in 1:(i-1))
      {
        result = result - L[i, j] * z[j]
      }
    }
    z[i] = result / L[i, i]
  }
  return(z)
}
```

(c)

```
backward = function(L, z)
{
  n = length(b)
  x = numeric(n)

  for (i in n:1)
  {
    result = z[i]

    if (i < n)
    {
      for(j in (i+1):n)
      {
        result = result - L[j, i] * x[j]
      }
    }
    x[i] = result / L[i, i]
  }
  return(x)
}
```

(d)

```
b = c(1, -2, 3)

z = forward(L, b)
x = backward(L, z)

print(x)
```

```
## [1] -0.0625 -0.6250  1.2500
```

```
print(solve(A, b))
```

```
## [1] -0.0625 -0.6250  1.2500
```

```
all.equal(x, solve(A, b))
```

```
## [1] TRUE
```

Problem 6

(a)

```
gaussian_kernel = function(x, x_prime, rho = 1)
{
  result = exp(-rho * abs(x  - x_prime)^2)

  return(result)
}
```

(b)

```
krr = function(X, y, lambda = 0.0001, rho = 1)
{
  n = nrow(X)
  K = matrix(0, n, n)

  for (i in 1:n)
  {
    for (j in 1:n)
    {
      K[i, j] = gaussian_kernel(X[i, 1], X[j, 1], rho)
    }
  }

  alpha = solve(K + lambda * diag(n), y)

  result = list(X = X, y = y, alpha = alpha, rho = rho)
  class(result) = "krr"

  return(result)
}
```

(c)

```
predict.krr = function(object, newdata)
{
  X = object$X
  alpha = object$alpha
  rho = object$rho

  n = nrow(X)
  m = nrow(newdata)

  pred = numeric(m)

  for (i in 1:m)
  {
    result = 0

    for(j in 1:n)
```

```
    {
      result = result + gaussian_kernel(newdata[i, 1], X[j, 1], rho) * alpha[j]
    }
    pred[i] = result
  }

  return(pred)
}
```

(d)

```
plot.krr = function(x)
{
  X = x$X
  y = x$y

  x_new = matrix(seq(min(X), max(X), length.out = 200), ncol = 1)
  y_pred = predict(x, x_new)

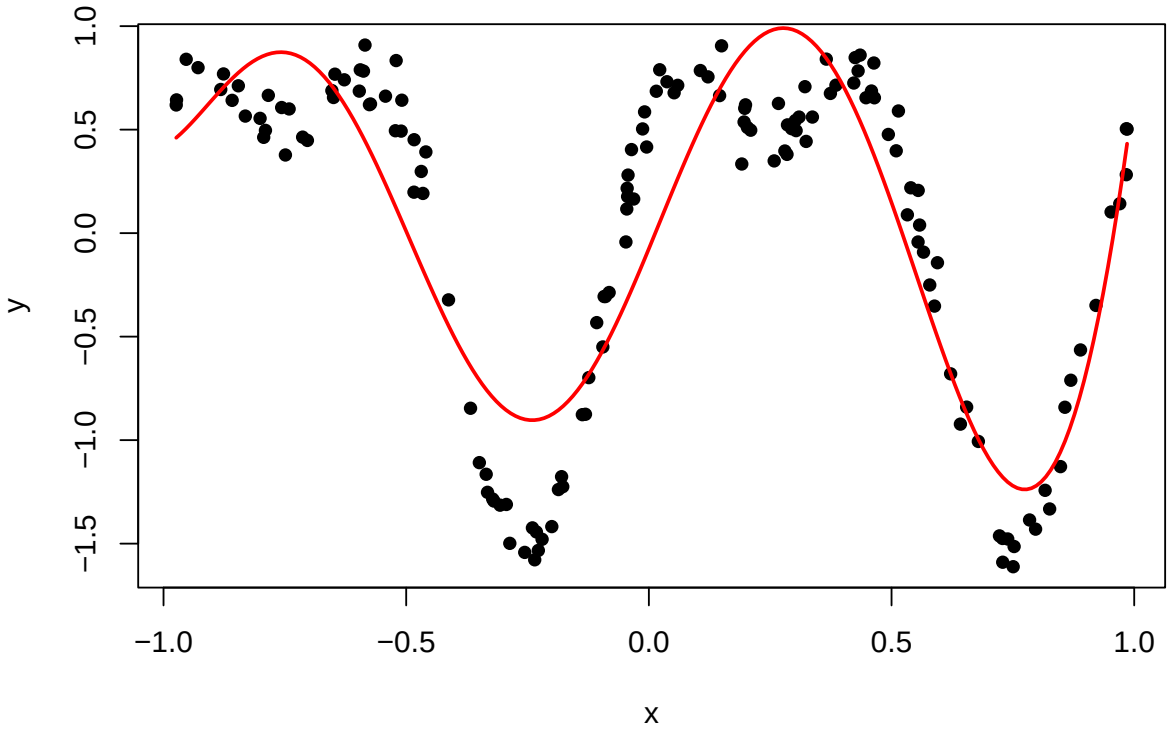
  plot(X[, 1], y, pch = 16, col = 'black',
       xlab = "x", ylab = "y")

  lines(x_new[, 1], y_pred, col = "red", lwd = 2)
}
```

(e)

```
set.seed(1)
n = 150
X = matrix(runif(n, -1, 1), ncol = 1)
ftrue = function(x) sin(2*pi*x) + 0.5 * cos(4*pi*x)
y = ftrue(X[,1]) + rnorm(n, sd = 0.1)

model = krr(X, y)
x_new = matrix(seq(-1, 1, length.out = 200), ncol = 1)
y_pred = predict(model, x_new)
plot(model)
```




```
png("plots/problem6.png", width = 1500, height = 1200)
plot(model)
dev.off()
```

```
## pdf
##   2
```